

Taller Práctico: API REST para Catálogo de Música

Objetivo: Desarrollar un backend completo desde cero utilizando **Node.js**, **TypeScript**, **Express** y **Prisma ORM** (con PostgreSQL) aplicando una arquitectura en capas. Crearán dos entidades fuertemente relacionadas: **Álbumes** y **Canciones**.

Requisitos de la Entrega (WhatsApp)

A lo largo del taller se solicitarán **3 capturas de pantalla** específicas que deberás enviar al grupo de WhatsApp del curso como evidencia de tu progreso.

Paso 1: Concepto de Composición y Configuración de Modelos

Explicación Teórica: ¿Qué es la Composición y el Borrado en Cascada?

En este taller utilizaremos una relación de **Composición** (o dependencia fuerte):

1. **Dependencia de existencia:** Una canción no puede existir sola; obligatoriamente debe pertenecer a un álbum. Por ende, el campo `albumId` en la canción **no puede ser nulo**.
2. **Borrado en Cascada (`onDelete: Cascade`):** Si borras un álbum de la base de datos, no tiene sentido dejar sus canciones "huérfanas" flotando en el sistema. Al activar el borrado en cascada, si eliminas un álbum, la base de datos **eliminará automáticamente** todas las canciones asociadas a él.

Acción: Actualizar el Schema de Prisma

1. Abre tu archivo `prisma/schema.prisma` (no borres el bloque de conexión actual de PostgreSQL).
2. Agrega los dos modelos configurando la relación en cascada:

None

```
model Album {
  id          Int          @id @default(autoincrement())
  titulo      String
  artista     String
  anio        Int
  canciones   Cancion[] // Un álbum tiene muchas canciones
}

model Cancion {
  id          Int          @id @default(autoincrement())
  titulo      String
  duracion    Int          // Duración en segundos (ej. 240)
  albumId     Int          // Clave foránea obligatoria
  // Establecemos la relación y definimos el borrado en cascada
  (onDelete: Cascade)
  album       Album        @relation(fields: [albumId], references: [id],
  onDelete: Cascade)
}
```

3. Ejecuta la migración en tu terminal:

Shell

```
npx prisma migrate dev --name agregar_albumes_y_canciones
```

CAPTURA 1 (WhatsApp)

Envía una captura donde se observe tu archivo `schema.prisma` con ambos modelos escritos y en la terminal el mensaje de éxito de la migración (`Your database is now in sync with your schema`).



Paso 2: Estructura del Proyecto

Asegúrate de crear los siguientes directorios y archivos dentro de tu carpeta `src`:

```
None
src/
├── database/
│   └── prisma.ts
├── controllers/
│   ├── album.controller.ts
│   └── cancion.controller.ts
├── routes/
│   ├── albums.ts
│   └── canciones.ts
└── index.ts
```

Paso 3: CRUD de Álbumes (Entidad 1)

Debes implementar los endpoints básicos para gestionar los álbumes de música.

JSONs de prueba para Álbumes en Postman

1. Crear Álbum (POST `http://localhost:3001/albums`)

- **Headers:** `Content-Type: application/json`
- **Body (raw - JSON):**

```
JSON
{
  "titulo": "Random Access Memories",
  "artista": "Daft Punk",
  "anio": 2013
}
```

2. Obtener Álbumes (GET `http://localhost:3001/albums`)

- **Respuesta esperada (JSON):**



JSON

```
[
  {
    "id": 1,
    "titulo": "Random Access Memories",
    "artista": "Daft Punk",
    "anio": 2013
  }
]
```

CAPTURA 2 (WhatsApp)

Envía una captura de Postman realizando una petición **POST** para crear un álbum y mostrando la respuesta exitosa con el estado `201 Created`.

Paso 4: CRUD de Canciones (Entidad 2)

Cada canción debe crearse apuntando al `id` de un álbum existente mediante el campo `albumId`.

1. En `src/controllers/cancion.controller.ts`, implementa:
 - `crearCancion` (POST)
 - `obtenerCanciones` (GET)
 - `eliminarCancion` (DELETE - por ID)
2. En `src/routes/canciones.ts`, enlaza los endpoints usando tu router.

JSONs de prueba para Canciones en Postman

1. Crear Canción (POST `http://localhost:3001/canciones`)
 - **Body (raw - JSON):** (Asegúrate de que el `albumId` corresponda a un ID de álbum que ya creaste)

JSON

```
{
  "titulo": "Instant Crush",
  "duracion": 337,
  "albumId": 1
}
```

2. Obtener Canciones (GET `http://localhost:3001/canciones`)
 - **Respuesta esperada (JSON):**



JSON

```
[
  {
    "id": 1,
    "titulo": "Instant Crush",
    "duracion": 337,
    "albumId": 1
  }
]
```

3. Eliminar Canción (DELETE <http://localhost:3001/canciones/1>)

- Respuesta esperada (JSON):

JSON

```
{
  "mensaje": "Canción eliminada correctamente"
}
```

CAPTURA 3 (WhatsApp)

Envía una captura de Postman haciendo un **GET** a <http://localhost:3001/canciones> donde se observe la lista de al menos **dos canciones diferentes** creadas por ti vinculadas a tu álbum.

Paso 5: Registro y Encendido del Servidor

En tu `src/index.ts`, debes registrar ambas rutas bajo los prefijos correspondientes:

- `/albumes` -> Router de álbumes
- `/canciones` -> Router de canciones

6. Entrega

- Link del Repositorio
- Capturas de Pantalla Enviadas al grupo de WhatsApp: